

POROČILO O RAZVOJU IN OPTIMIZACIJI MODELA

1. Izbira modelov in algoritmov (Uvod in kontekst problema)

V sklopu projekta Pametni paketnik smo implementirali dvostopenjsko avtentikacijo (2FA) s prepoznavo obraza (Face ID). Za jedro sistema smo izbrali pristop prenosnega učenja (Transfer Learning) z uporabo konvolucijske nevronske mreže **ResNet18**, ki je bila vnaprej naučena na obsežni podatkovni zbirki **ImageNet**.

Zakaj ResNet18? Arhitektura ResNet (Residual Networks) z uporabo preskočnih povezav (skip connections) učinkovito rešuje problem izginjajočega gradienta v globokih mrežah. Različica z 18 plastmi ponuja odličen kompromis med računsko zahtevnostjo (hitrostjo učenja na omejenih virih) in natančnostjo modela, kar je idealno za izvajanje na vgrajenih sistemih in mobilnih napravah.

Arhitekturne prilagoditve: Ker je originalni model grajen za klasifikacijo 1000 splošnih razredov, smo zadnjo polno povezano plast (fully connected layer) odstranili ter jo nadomestili z novo linearno plastjo (**model.fc**), prilagojeno številu uporabnikov v našem sistemu. Globoke sloje mreže smo zamrznili, da so delovali kot robusten ekstraktor vizualnih značilnosti, učili pa smo le novozgrajeni klasifikacijski sloj.

Začetni problem v realnem okolju

Med testiranjem sistema v živo preko Android aplikacije smo naleteli na kritično težavo: sistem je kljub izjemno visokim rezultatom natančnosti med učenjem na računalniku v realnem okolju konstantno zamenjeval osebe (npr. uporabnika je prepoznal kot napačnega sošolca in mu odobril napačne pravice) ali pa je povsem odpovedal ob minimalni spremembi svetlobe. Zaradi tega smo morali izvesti večfazno optimizacijo hiperparametrov, da bi ugotovili, zakaj matematični ideal na papirju v praksi odpove.

2. Optimizacija hiperparametrov in potek učenja

Učenje nevronske mreže je potekalo iterativno. Da bi našli najbolj stabilno konfiguracijo, smo izvedli sistematsko iskanje po mreži (Grid Search) naslednjih hiperparametrov:

- **Stopnja učenja (Learning Rate - LR) [0.01, 0.001, 0.0001]:** Določa velikost koraka pri popravljanju uteži. Vrednost 0.01 je bila izbrana kot višja stopnja (pomembna za SGD), 0.001 kot industrijski standard, 0.0001 pa kot nizka stopnja, primerna za fine nastavitve (fine-tuning) vnaprej naučenih mrež, kjer ne želimo uničiti že naučenih filtrov.
- **Velikost serije (Batch Size) [32, 64, 128]:** Določa število slik, ki jih mreža procesira pred posodobitvijo uteži. Manjše serije (32) vnašajo več regularizacijskega šuma, večje (64 in 128) pa ponujajo stabilnejšo oceno gradienta in boljšo izrabo strojne opreme (GPU).
- **Število epoh [5, 10, 12]:** Ena epoha pomeni, da je mreža enkrat prehodila celotno učno množico. Testirali smo krajše cikle (5 epoh) za hitro preverjanje konvergence ter daljše (10 in 12), da modelu omogočimo stabilizacijo natančnosti brez prevelikega prilagajanja (overfittinga).

- **Optimizatorja (Adam in SGD):** Primerjali smo adaptivni Adam (ki samostojno prilagaja stopnjo učenja za vsak parameter in hitro konvergira) ter klasični Stohastični gradientni spust (SGD), ki uteži posodablja enakomerno in linearno.

2.1. Analiza poskusov in argumentacija izbire (Faza 1)

Poskus 1: Adam z visoko stopnjo učenja (LR = 0.01, Batch Size = 32)

Rezultati iz dnevnika: Učna natančnost: 99.01% | Validacijska natančnost: 100.00%

Učna izguba: 0.0817, Validacijska izguba: 0.0703

Analiza in ugotovitve: Grafi izgube in natančnosti kažejo izrazito nestabilnost (cik-cak skoki okoli 4. in 5. epohe). Ker je algoritem Adam adaptiven, je v kombinaciji s previsoko stopnjo učenja (LR = 0.01) izvajal preveč agresivne popravke uteži.

Primernost za aplikacijo: Popolnoma neprimerno. Čeprav končne številke izgledajo visoke, ostri skoki na grafu nakazujejo, da je model začel razvijati »teoretično iluzijo«. Namesto robustnih obraznih potez se je naučil specifičnega šuma ozadja in trenutne svetlobe. V realnem okolju aplikacije je to povzročilo lažno 100-% samozavest in kritično zamenjevanje identitet razvijalcev.

Poskus 2: Adam z vmesno stopnjo učenja (LR = 0.001, Batch Size = 32)

Rezultati iz dnevnika: Učna natančnost: 99.40% | Validacijska natančnost: 100.00%

Učna izguba: 0.0595, Validacijska izguba: 0.0521

Analiza in ugotovitve: Ta poskus predstavlja industrijski standard. Pričakovano je prinesel hitro konvergenco in navidezno stabilizacijo krivulje natančnosti blizu vrha že po nekaj epohah. Vendar pa se je v našem specifičnem primeru (majhnih zbirk slik uporabnikov za Face ID) prav tako izkazal za past.

Primernost za aplikacijo: Delno neuspešno za produkcijo. Adam je s svojo adaptivno naravo prehitro našel matematične bližnjice v našem "zaprtem svetu" treh razredov. Model je dosegel visoko natančnost na validacijskih slikah iz iste mape, v realnem času pred kamero telefona pa je odpovedal, saj je bil še vedno preveč občutljiv na piksele ozadja in ni razvil prave generalizacije obraza.

Poskus 3: Adam z nizko stopnjo učenja (LR = 0.0001, Batch Size = 32)

Rezultati iz dnevnika: Učna natančnost: 95.73% | Validacijska natančnost: 91.24%

Učna izguba: 0.3956, Validacijska izguba: 0.4606

Analiza in ugotovitve: Krivulji izgube in natančnosti sta maksimalno zglajeni in stabilni, brez kakršnih koli nihanj. Vendar pa je napredek zaradi izjemno majhnih, mikroskopskih korakov posodabljanja uteži prepočasen. Graf jasno nakazuje, da bi model za doseganje polnega potenciala potreboval bistveno več kot le 10 epoh.

Primernost za aplikacijo: Neprimerno za trenutno fazo. Model je v tem stanju izrazito podtreniran. Validacijska natančnost 91.24 % in visoka izguba pomenita, da bi Face ID v Android aplikaciji prepogosto zavračal prave uporabnike, kar bi vodilo v slabo uporabniško izkušnjo.

Poskus 4: SGD z visoko stopnjo učenja (LR = 0.01, Batch Size = 32)

Rezultati iz dnevnika: Učna natančnost: 99.01% | Validacijska natančnost: 100.00%

Učna izguba: 0.0817, Validacijska izguba: 0.0703

Analiza in ugotovitve: Ta stopnja učenja se je za SGD izkazala za idealno. Ker SGD nima lastnega pospeševanja (kot Adam), potrebuje dovolj velike korake (LR = 0.01), da naredi opazen napredek. Krivulja izgube lepo in zanesljivo pada, natančnost pa stabilno raste skozi vseh 10 epoh. Prisoten je minimalen šum (cik-cak skoki), kar je za majhen paket (32) povsem običajno in celo koristno, saj deluje kot blaga regularizacija.

Primernost za aplikacijo: Izjemno primerno. Model dosega vrhunsko natančnost, hkrati pa (za razliko od Adama) ne ustvarja lažne samozavesti na podlagi pikslov ozadja. Uteži se posodablajo enakomerno, kar prisili mrežo v učenje dejanskih obraznih potez.

Poskus 5: SGD z vmesno stopnjo učenja (LR = 0.001, Batch Size = 32)

Rezultati iz dnevnika: Učna natančnost: 95.93% | Validacijska natančnost: 94.47%

Učna izguba: 0.3695, Validacijska izguba: 0.4030

Analiza in ugotovitve: Model izvaja 10-krat manjše in bolj previdne korake. Krivulja izgube pada izjemno gladko in z manj nihanja kot pri LR 0.01. Vendar pa zaradi odsotnosti dinamičnega pospeševanja graf natančnosti narašča opazno počasneje.

Primernost za aplikacijo: Delno neprimerno za kratke cikle. Čeprav model lepo konvergira in kaže dobro generalizacijo (krivulji sta tesno skupaj), je po 10 epohah še vedno preveč previden. Validacijska natančnost 94.47% pomeni, da bi v produkciji v živo na preveč obrazih naletelo na zavrnitev dostopa.

Poskus 6: SGD z nizko stopnjo učenja (LR = 0.0001, Batch Size = 32)

Rezultati iz dnevnika: Učna natančnost: 65.04% | Validacijska natančnost: 62.21%

Učna izguba: 0.9047, Validacijska izguba: 0.9235

Analiza in ugotovitve: Šolski primer podtreniranosti (underfittinga). Ker model začenja trenirati iz nič (zadnji sloj), so mikroskopski koraki pri LR = 0.0001 enostavno premajhni, da bi se model v 10 epohah premaknil proti optimumu. Krivulja izgube se komaj spušča, natančnost pri 62.21% pa je le malo nad naključnim ugibanjem.

Primernost za aplikacijo: Popolnoma neuporabno. S tem modelom Face ID v aplikaciji sploh ne bi deloval, saj bi sistem konstantno zavračal vse legitimne uporabnike. Ta poskus empirično dokazuje, da SGD nujno potrebuje višji LR ali pa drastično večje število epoh.

Poskus 7: Adam z visoko stopnjo učenja (LR = 0.01, Batch Size = 64)

Rezultati iz dnevnika: Učna natančnost: 99.50 % | Validacijska natančnost: 100.00 %

Učna izguba: 0.0215, Validacijska izguba: 0.0269

Analiza in ugotovitve: Večja velikost paketa (64) je ublažila ostre cik-cak skoke, ki smo jih opazili pri paketu 32. Krivulje so opazno bolj gladke. Vendar pa nevarnost agresivnega preučenja (overfittings) ostaja izjemno visoka. Model je zaradi previsoke stopnje učenja že v prvi epohi skočil na skoraj maksimalno natančnost.

Primernost za aplikacijo: Neprimerno. Čeprav je graf stabilnejši kot prej, se je model zaradi kombinacije $LR = 0.01$ in hitre konvergence preprosto "piflal" vzorce ozadja in svetlobe namesto dejanskih obraznih potez, kar je v živo znova vodilo v zamenjevanje identitet.

Poskus 8: Adam z vmesno stopnjo učenja ($LR = 0.001$, Batch Size = 64)

Rezultati iz dnevnika: Učna natančnost: 99.30 % | Validacijska natančnost: 100.00 %

Učna izguba: 0.0883, Validacijska izguba: 0.0988

Analiza in ugotovitve: Ta poskus je na papirju izgledal kot idealen kandidat za zmagovalni model. Krivulja izgube pada izjemno lepo, gladko in brez kakršnih koli šumnih skokov, natančnost pa hitro doseže stabilen plato. Kombinacija stabilnejšega paketa (64) in optimalnega koraka (0.001) je teoretično zagotavljala odlično učenje.

Primernost za aplikacijo: Varnostna iluzija (Zavrnjeno za produkcijo). Kljub "šolskim" krivuljam na grafu je prav ta poskus dokazal past zaprtega sveta. Adam je brez težav dosegel 100% validacijsko natančnost, vendar je v realnem svetu pred kamero telefona popolnoma odpovedal. Ker je mreža prehitro in preveč suvereno zaključila učenje, ni razvila tolerance na spremembe okolja, ampak je vračala lažno visoko samozavest za napačne osebe.

Poskus 9: Adam z nizko stopnjo učenja ($LR = 0.0001$, Batch Size = 64)

Rezultati iz dnevnika: Učna natančnost: 92.25% | Validacijska natančnost: 88.02%

Učna izguba: 0.5709, Validacijska izguba: 0.5974

Analiza in ugotovitve: Krivulje so popolnoma gladke in stabilne, kar potrjuje, da večji paket v kombinaciji z majhnimi koraki uspešno odstrani ves šum. Vendar pa je pri $LR = 0.0001$ število posodobitev uteži na epoko premajhno. Napredek je linearen, a prepočasen, zato je 10 epoh premalo, da bi iztisnili maksimum iz mreže.

Primernost za aplikacijo: Neprimerno (Podtreniranost). Validacijska natančnost pod 88% pomeni, da bi model v realnem času zavračal skoraj vsakega drugega legitimnega uporabnika, kar bi povsem uničilo uporabniško izkušnjo Android aplikacije.

Poskus 10: SGD z visoko stopnjo učenja ($LR = 0.01$, Batch Size = 64)

Rezultati iz dnevnika: Učna natančnost: 99.70% | Validacijska natančnost: 99.54%

Učna izguba: 0.1072, Validacijska izguba: 0.1261

Analiza in ugotovitve: Povečanje velikosti paketa na 64 je v kombinaciji s stopnjo učenja 0.01 prineslo izjemno stabilizacijo učenja. Krivulji izgube (Loss) in natančnosti (Accuracy) sta opazno bolj gladki in znatno manj nihata kot pri Poskusu 4 (kjer je bil Batch Size 32). Model lepo in enakomerno konvergira skozi vseh 10 epoh. Večji paket deluje kot naravna zaščita pred prehitrim preučenjem.

Primernost za aplikacijo: Odlična produkcijska osnova. Ta poskus ponuja idealno razmerje med visoko natančnostjo in stabilnostjo gradienta. Ker gre za SGD, se model ne zanaša na trenutne bližnjice v ozadju, ampak enakomerno posodablja uteži na podlagi dejanskih obraznih potez.

Poskus 11: SGD z vmesno stopnjo učenja (LR = 0.001, Batch Size = 64)

Rezultati iz dnevnika: Učna natančnost: 93.55% | Validacijska natančnost: 88.48%

Učna izguba: 0.5490, Validacijska izguba: 0.6019

Analiza in ugotovitve: Krivulje so estetsko zelo lepe, stabilne in enakomerno padajo oziroma rastejo. Ker pa klasični SGD nima notranjega pospeška, se pri večjem paketu (64) število posodobitev uteži znotraj epohe razpolovi. Napredek je opazno upočasnen.

Primernost za aplikacijo: Podtrenirano za 10 epoh. Validacijska natančnost 88.48 % je prenizka za produkcijo. Ta graf jasno dokazuje, da bi model pri tej konfiguraciji nujno potreboval precej več kot le 10 epoh, da bi razvil svoj polni potencial in dosegel visoko stopnjo zaupanja.

Poskus 12: SGD z nizko stopnjo učenja (LR = 0.0001, Batch Size = 64)

Rezultati iz dnevnika: Učna natančnost: 42.80 % | Validacijska natančnost: 37.79 %

Učna izguba: 1.0467, Validacijska izguba: 1.0874

Analiza in ugotovitve: Najizrazitejši primer podučnosti (underfittinga) v celotnem testiranju. Kombinacija velikega paketa (manj posodobitev) in mikroskopsko majhne stopnje učenja pomeni, da se model skorajda ni premaknil z izhodiščne točke. Krivulja izgube ostaja izjemno visoko, natančnost pri 37.79 % na validaciji pa je povsem blizu naključnemu ugibanju.

Primernost za aplikacijo: Popolnoma neuporabno. Ta poskus je služil kot ključen empirični dokaz, da je klasični SGD pri nizkih stopnjah učenja brez drastičnega povečanja števila epoh popolnoma nemočen.

Poskus 13: Adam z visoko stopnjo učenja (LR = 0.01, Batch Size = 128)

Rezultati iz dnevnika: Učna natančnost: 99.60% | Validacijska natančnost: 100.00%

Učna izguba: 0.0258, Validacijska izguba: 0.0201

Analiza in ugotovitve: Izjemno velik paket (128) je maksimalno stabiliziral ocene gradientov. Krivulje so popolnoma gladke, brez kakršnih koli šumnih nihanj. Vendar pa graf natančnosti razkriva znano težavo: model že v drugi epohi doseže plato blizu 100%. Adam je s pomočjo lastnega momenta nadoknadil majhno število posodobitev in znova zgradil teoretično iluzijo.

Primernost za aplikacijo: Nevarno. Gladek graf je v tem primeru vizualna prevara. Ker gre za zaprt svet treh razredov, je model z izjemno agresivnimi koraki prehitro dosegel lokalni minimum tako, da se je preučil (overfitting) na ozadje, kar je v živo povzročalo lažno samozavest in napačno spuščanje oseb skozi Face ID.

Poskus 14: Adam z vmesno stopnjo učenja (LR = 0.001, Batch Size = 128)

Rezultati iz dnevnika: Učna natančnost: 99.60 % | Validacijska natančnost: 98.62 %

Učna izguba: 0.1128, Validacijska izguba: 0.1622

Analiza in ugotovitve: Izjemno velika velikost paketa (128) poskrbi, da so ocene gradientov maksimalno stabilne. Krivulji izgube in natančnosti sta estetsko čisti, brez kakršnih koli šumnih cik-cak skokov. Vendar pa graf natančnosti razkriva, da model že okoli 4. epohe doseže stabilen plato tik pod vrhom. Kljub teoretično dobri generalizaciji pa za naš projekt to še vedno pomeni težavo.

Primernost za aplikacijo: Rizično (Zavrnjeno). Čeprav večji paket ublaži najhujši šum, Adam pri stopnji 0.001 zaradi lastnega momenta v zaprtem okolju prehitro reši matematični problem. Model se zanaša na globalne piksele slike (ozadje in svetlobo), namesto da bi natančno izoliral obrazne poteze, kar v realnem delovanju povzroči varnostno ranljivost.

Poskus 15: Adam z nizko stopnjo učenja (LR = 0.0001, Batch Size = 128)

Rezultati iz dnevnika: Učna natančnost: 86.69 % | Validacijska natančnost: 78.80 %

Učna izguba: 0.7242, Validacijska izguba: 0.7772

Analiza in ugotovitve: Model izvaja mikroskopske korake posodabljanja (LR = 0.0001), hkrati pa ima zaradi velikega paketa (128) minimalno število posodobitev uteži na posamezno epoko. Krivulja izgube pada popolnoma linearno in gladko, vendar prepočasno. Graf natančnosti jasno kaže, da je 10 epoh drastično premalo, saj model ob koncu učenja šele počasi leze proti optimumu.

Primernost za aplikacijo: Popolnoma neuporabno (Podtreniranost). Validacijska natančnost 78.80 % pomeni, da bi Face ID v aplikaciji konstantno zavračal legitimne uporabnike. Model je v tem stanju preveč neprepričan in matematično nerazvit.

Poskus 16: SGD z visoko stopnjo učenja (LR = 0.01, Batch Size = 128)

Rezultati iz dnevnika: Učna natančnost: 99.21 % | Validacijska natančnost: 98.62 %

Učna izguba: 0.1763, Validacijska izguba: 0.2388

Analiza in ugotovitve: Ta konfiguracija je prinesla izjemno lepe rezultate. Kombinacija razmeroma visoke stopnje učenja (0.01) uspešno kompenzira majhno število posodobitev uteži na epoko, ki jih prinaša velik paket. Krivulje na grafih so lepo zglajene, brez kakršnih koli nihanj ali šumnih cik-cak skokov. Paket 128 tukaj deluje kot odličen stabilizator učenja.

Primernost za aplikacijo: Model lepo in enakomerno raste brez znakov prehitrega preučenja (overfittinga). Vendar pa zaradi velikega paketa končna izguba ostaja nekoliko višja (0.2388 na validaciji) kot pri zmagovalnem modelu z batch size 64, kar pomeni, da bi za polno suverenost pred kamero model potreboval več kot 10 epoh.

Poskus 17: SGD z vmesno stopnjo učenja (LR = 0.001, Batch Size = 128)

Rezultati iz dnevnika: Učna natančnost: 74.58 % | Validacijska natančnost: 71.89 %

Učna izguba: 0.7907, Validacijska izguba: 0.8045

Analiza in ugotovitve: Šolski primer podučnosti (underfittinga) zaradi premočne upočasnitve. Ker klasični SGD nima notranjega pospeška (momenta), se model ob kombinaciji previdnih korakov

(0.001) in minimalnega števila posodobitev na epoho premika izjemno počasi. Krivulja izgube sicer estetsko in gladko pada, vendar je naklon padanja izrazito prenizek.

Primernost za aplikacijo: Neprimerno. Validacijska natančnost 71.89 % po 10 epohah jasno dokazuje, da bi model v tej kombinaciji potreboval stotine epoh, da bi sploh dosegel uporabno raven za prepoznavo obrazov.

Poskus 18: SGD z nizko stopnjo učenja (LR = 0.0001, Batch Size = 128)

Rezultati iz dnevnika: Učna natančnost: 33.66 % | Validacijska natančnost: 28.57 %

Učna izguba: 1.1132, Validacijska izguba: 1.2024

Analiza in ugotovitve: To je najbolj ekstremna in najpočasnejša možna konfiguracija v celotnem projektu. Model se zaradi mikroskopskih korakov (0.0001) praktično sploh ni premaknil z izhodiščne točke. Natančnost na validaciji (28.57%) je celo padla pod mejo naključnega ugibanja, graf izgube pa izgleda skoraj kot ravna vodoravna črta.

Primernost za aplikacijo: Popolnoma neuporabno. Poskus služi kot odličen empirični dokaz ekstremnih matematičnih omejitev algoritma SGD, ko mu odvzamemo tako frekvenco posodobitev kot dolžino koraka.

2.1.1. Odločitev izbire hiperparametrov

Izbira optimizatorja: Adam proti SGD (Dinamičnost vs. Konsistentnost)

Preizkus dveh popolnoma različnih filozofij optimizacije je bil ključen za varnost našega Face ID sistema:

Kaj smo preizkušali z Adamom: Adam je prilagodljiv algoritem, ki samodejno določa hitrost učenja za vsako utež posebej. Z njim smo želeli preveriti, ali lahko z njegovo hitro konvergenco dosežemo stabilen model v izjemno kratkem času (le 10 epoh).

Kaj smo preizkušali s SGD: SGD (Stochastic Gradient Descent) je klasičen optimizator brez notranjega pospeševanja, ki uteži posodablja enakomerno. Z njim smo želeli dokazati hipotezo, da bo počasnejše, a bolj togo učenje prisililo mrežo v iskanje splošnih obraznih potez namesto okoljskih bližnjic.

Kaj smo dokazali: Dokazali smo "past zaprtega sveta". Adam je na papirju blestel (hitro dosegal 100 % natančnost), a je v živo odpovedal, ker se je preučil (overfitting) na šum ozadja in svetlobo. SGD se je izkazal za varnejšega, saj ne išče statističnih bližnjic.

Stopnja učenja (Learning Rate): 0.01, 0.001 in 0.0001 (Velikost koraka)

Stopnja učenja določa, kako močno se spremenijo uteži modela ob odkriti napaki.

Visok LR (0.01): Želeli smo preveriti, ali model lahko hitro preskoči začetne lokalne minimume. Dokazali smo, da je za SGD to idealna stopnja za zagon učenja, pri Adamu na majhnem paketu pa povzroči divje nihanje (cik-cak grafe).

Vmesni LR (0.001): Iskali smo zlato sredino, ki bi ponudila stabilen in gladek padec izgube.

Nizek LR (0.0001): Želeli smo preveriti teorijo finega uravnavanja (fine-tuning). Dokazali smo, da so pri 10 epohah mikroskopski koraki popolnoma nemočni, kar je vodilo v hudo podtreniranost (underfitting), kjer je bila natančnost blizu naključnega ugibanja.

Velikost paketa (Batch Size): 32, 64 in 128 (Stabilnost gradienta)

Velikost paketa določa, koliko slik si model ogleda hkrati, preden enkrat posodobi svoje uteži.

Paket 32: Vnaša več statističnega šuma (ker ocenjuje gradient na manj vzorcih), kar lahko pomaga modelu, da ne nasede v lokalnih minimumih. Grafi so bili tukaj najbolj nemirni.

Paket 64 in 128: S povečevanjem paketa smo želeli stabilizirati učenje. Dokazali smo, da večji paket deluje kot vizualni likalnik grafov – krivulje izgube in natančnosti so postale popolnoma gladke in linearne. Povečanje paketa hkrati pomeni manjše število korakov (posodobitev) znotraj ene epohe. Dokazali smo, da če povečamo paket na 64 ali 128, moramo nujno zvišati stopnjo učenja ali podaljšati število epoh, sicer model ostane podučen.

Zakaj ravno omejitve na 10 epoh? Dolžino učenja smo namenoma zaklenili na 10 epoh za vse poskuse, ker smo s tem ustvarili enake začetne pogoje (baseline). 10 epoh je idealno okno, ki nam hitro razkrije: Dinamiko konvergence (kako hitro model dojema podatke). Nagnjenost k preučenju (če gre graf takoj na 100 %, vemo, da je algoritem preveč agresiven).

2.2. Analiza poskusov in argumentacija izbire (Faza 2)

Poskus 19: Adam z visoko stopnjo učenja (LR = 0.01, Batch Size = 128, Epohe: 12)

Rezultati iz dnevnika: Učna natančnost: 99.40% | Validacijska natančnost: 100.00%

Učna izguba: 0.0215, Validacijska izguba: 0.0252

Analiza in ugotovitve: Ta poskus je služil kot test stabilnosti pri agresivni stopnji učenja (LR = 0.01) na daljši rok. Kljub začetnemu tveganju za preučenost zaradi kombinacije visokega LR in 12 epoh, grafi kažejo, da je model ostal izjemno stabilen. Validacijska izguba ni začela ponovno rasti, ampak je skupaj z učno izgubo padla blizu ničle. Na grafu natančnosti opazimo manjše nihanje na začetku (okoli 2. epohe), nato pa krivulji gladko konvergirata proti vrhu.

Primernost za aplikacijo: Izjemno visoka. Model dosega perfektno natančnost na validacijski množici brez znakov preučenosti, kar pomeni, da je zelo zanesljiv.

Poskus 20: Adam z visoko stopnjo učenja (LR = 0.01, Batch Size = 128, Epohe: 5)

Rezultati iz dnevnika: Učna natančnost: 99.60% | Validacijska natančnost: 99.54%

Učna izguba: 0.0461, Validacijska izguba: 0.0520

Analiza in ugotovitve: Ta poskus je testiral hitrost konvergence. Zaradi stabilnega in matematično "čistega" gradienta, ki ga omogoča velik paket (128), si je model lahko privoščil agresiven LR = 0.01. Rezultati potrjujejo hipotezo: model je dosegel vrhunski rezultat že v polovičnem času (pri 5 epohah). Krivulja natančnosti bliskovito naraste in se stabilizira že po 2. epohi, izguba pa se hitro spusti proti ničli.

Primernost za aplikacijo: Odlična. Model je izjemno natančen in učinkovit, saj doseže skoraj popoln rezultat z minimalno porabo časa in računske moči (le 5 epoh).

Poskus 21: Adam z vmesno stopnjo učenja (LR = 0.001, Batch Size = 32, Epohe: 5)

Rezultati iz dnevnika: Učna natančnost: 99.21% | Validacijska natančnost: 97.70%

Učna izguba: 0.1173, Validacijska izguba: 0.1301

Analiza in ugotovitve: Namen poskusa je bil preveriti, ali lahko manjši paket (32), ki prinaša več statističnega šuma in več popravkov na epoho, kompenzira nižjo, a varnejšo stopnjo učenja (LR = 0.001). V le 5 epohah je model dosegel zelo visoko natančnost, vendar krivulja izgube še vedno kaže strm padajoči trend. To pomeni, da se model uči pravilno, vendar je bil proces prekinjen prehitro, preden bi dosegel svoj polni potencial.

Primernost za aplikacijo: Delno uporabno. Natančnost 97.70% je dobra, vendar model še ni povsem dokončal učenja, zato izguba ostaja relativno visoka (0.1301).

Poskus 22: Adam z vmesno stopnjo učenja (LR: 0.001, Batch Size: 32, Epohe: 12)

Rezultati iz dnevnika: Učna natančnost: 99.40% | Validacijska natančnost: 100.00%

Učna izguba: 0.0528, Validacijska izguba: 0.0427

Analiza in ugotovitve: Tukaj smo preverjali končni domet teoretično najbolj optimalne in varne konfiguracije (LR = 0.001 in Batch Size = 32) skozi 12 epoh. Ker je korak manjši, graf natančnosti kaže več nihanja (cik-cak vzorec med 2. in 6. epoho) zaradi šuma manjšega paketa. Vendar pa z daljšim časom učenja model doseže popoln plato. Validacijska izguba (0.0427) je celo nižja od učne izgube (0.0528), kar pomeni, da je model idealno natreniran, odlično posplošuje in se ni preučil.

Primernost za aplikacijo: Popolna. To je šolski primer idealno uravnoteženega modela, ki dosega 100% natančnost na validacijskih podatkih ob zelo nizki in stabilni izgubi.

Poskus 23: SGD z visoko stopnjo učenja (LR: 0.01, Batch Size: 32, Epohe: 5)

Rezultati iz dnevnika: Učna natančnost: 98.51% | Validacijska natančnost: 100.00%

Učna izguba: 0.1326, Validacijska izguba: 0.1251

Analiza in ugotovitve: Ker je paket majhen (32), model izvede veliko število posodobitev uteži na epoho, kar ob kombinaciji z visokim LR = 0.01 ustvari izjemno dinamično in hitro učenje. Krivulja izgube strmo in enakomerno pada, kar kaže, da visoka frekvenca korakov učinkovito nadomešča odsotnost prilagodljivih mehanizmov (ki jih ima npr. Adam). Validacijska natančnost že pri 4. epohi doseže 100%, vendar razmeroma visoka končna izguba (0.1251) opozarja, da proces učenja še ni povsem zaključen in da so uteži še vedno v fazi grobega prilagajanja.

Primernost za aplikacijo: Delno uporabno. Čeprav je validacijska natančnost navidez popolna, visoka izguba pomeni, da model še ni stabiliziran na varnem platoju.

Poskus 23: SGD z visoko stopnjo učenja (LR: 0.01, Batch Size: 32, Epohe: 12)

Rezultati iz dnevnika: Učna natančnost: 98.41% | Validacijska natančnost: 100.00%

Učna izguba: 0.0873, Validacijska izguba: 0.0655

Analiza in ugotovitve: Ta poskus predstavlja najbolj nemirno konfiguracijo v tej fazi zaradi kombinacije majhnega paketa (ki vnaša veliko statističnega šuma) in dolgega teka 12 epoh. Na grafu se to jasno odraža kot opazno cik-cak nihanje med 2. in 6. epoho. Vendar pa prav ta dinamičnost prepreči, da bi se model ujel v lokalne minime. Validacijska izguba proti koncu gladko pade pod učno izgubo ($0.0655 < 0.0873$). Kljub agresivnemu učenju model ob koncu ne kaže znakov izgube natančnosti ali preučenosti.

Primernost za aplikacijo: Dolg čas učenja je uspešno ukrotil začetni šum in model pripeljal do varnega, visoko natančnega in stabilnega stanja.

Poskus 24: SGD z visoko stopnjo učenja (LR: 0.01, Batch Size: 64, Epohe: 5)

Rezultati iz dnevnika: Učna natančnost: 98.81% | Validacijska natančnost: 99.08%

Učna izguba: 0.1893, Validacijska izguba: 0.2066

Analiza in ugotovitve: S povečanjem paketa na 64 smo poskušali najti idealno ravnovesje med stabilnostjo gradienta in šumnostjo. Ker klasični SGD nima dinamičnega pospeška, grafi potrjujejo teoretična pričakovanja: model zaradi linearnega načina učenja potrebuje več časa za "ogrevanje uteži". Krivulja izgube pada lepo in zvezno, brez kakršnih koli nihanj, vendar je končna izguba (0.2066) še vedno previsoka.

Primernost za aplikacijo: Slaba. Čeprav je natančnost blizu 99%, je izguba previsoka za zanesljivo delovanje v realnem času, saj model še ni dovolj natančno izpilil svojih meja odločanja.

Poskus 24: SGD z visoko stopnjo učenja (LR: 0.01, Batch Size: 64, Epohe: 12)

Rezultati iz dnevnika: Učna natančnost: 98.91% | Validacijska natančnost: 99.08%

Učna izguba: 0.1105, Validacijska izguba: 0.1192

Analiza in ugotovitve: Podaljšanje učenja na 12 epoh pri paketu 64 je ponudilo modelu idealno priložnost, da doseže svoj polni potencial. Krivulji izgube na grafu padata izjemno enakomerno in vzporedno skozi celoten proces, kar potrjuje ključno teoretično prednost: SGD je zaradi enakomernega koraka bistveno bolj varen pred preučenostjo kot Adam. Kljub temu pa opazimo, da se je validacijska natančnost po 6. epohi popolnoma ustavila na platoju 99.08% in izguba ne pada več bistveno.

Primernost za aplikacijo: Zelo dobra in varna. Model morda ne dosega absolutnih 100% kot pri manjših paketih, vendar sta stabilnost krivulj in odpornost proti preučenosti šolski primer zanesljivega in robustnega modela.

2.2.1. Razlaga in argumentacija ciljev 2. faze testiranja

Z izvedbo druge faze testiranja smo želeli nadgraditi statične ugotovitve prve faze in dokazati tri ključne vidike delovanja modela: časovno stabilnost, hitrost konvergence in odpornost na preučenje (overfitting).

Medtem ko nam je 1. faza pri fiksno zaklenjenih 10 epohah služila kot grobo sito za eliminacijo neuporabnih konfiguracij (predvsem modelov s prenizko stopnjo učenja, ki so povzročili hudo podtreniranost), smo v 2. fazi z variacijo dolžine učenja (5 in 12 epoh) testirali dinamično vedenje preostalih favoritov.

Z 2. fazo smo želeli dokazati naslednje hipoteze:

Obstoj in nevarnost "teoretične iluzije" pri hitri konvergenci (5 epoh): Želeli smo preveriti, ali lahko agresivnejše konfiguracije (kombinacija algoritma Adam, visokega LR in velikega Batch Size) dosežejo robusten optimum v ekstremno kratkem času. Dokazali smo, da nekateri modeli (npr. Poskus 20) bliskovito dosežejo visoko natančnost, vendar s tem tvegajo površinsko učenje (piksli ozadja), če se proces prekine prehitro, preden se izguba (Loss) stabilizira na dejanskem minimumu.

Dolgoročna stabilnost in odpornost proti preučanju (12 epoh): Pri podaljšanju učenja na 12 epoh je bil cilj ugotoviti, ali bosta Adam in SGD optimizatorja ob daljši izpostavljenosti podatkom začela kazati znake preučenosti (kjer bi validacijska izguba začela ponovno rasti, natančnost pa padati). Dokazali smo ključno razliko v njuni filozofiji: Adam teži k hitremu zaklepanju v lokalne minimume zaprtega sveta, medtem ko je SGD pri 12 epohah (Poskus 24) pokazal šolski primer varne linearne konvergence, kjer krivulje ostanejo vzporedne, model pa postane imun na zunanje šume okolja.

Matematična kompenzacija med velikostjo paketa (Batch Size) in časom učenja: Želeli smo dokazati, da lahko manjši paket (32), ki zaradi statističnega šuma povzroča nemirne "cik-cak" grafe, z zadostnim številom epoh (12) popolnoma zgladi svoje nihanje in doseže vrhunsko generalizacijo (kot v Poskusu 22). Obratno pa večji paketi (64 in 128) potrebujejo časovni podaljšek, da nadoknadijo manjše število posodobitev uteži na epoko.

Zaključek faze: Druga faza je potrdila, da za končno implementacijo v realno okolje Android aplikacije ni pomembna le visoka končna številka natančnosti, temveč oblika krivulje skozi čas. Dokazali smo, da stabilna validacijska izguba, ki v daljšem časovnem oknu (12 epoh) pade pod učno izgubo, pomeni edino pravo jamstvo, da bo Face ID sistem zanesljivo prepoznaval obraze tudi ob spremenjeni svetlobi in v novem okolju.

3. Utemeljitev končne izbire modela (SGD, 30 epoh, Batch 64, LR 0.01)

Za končni produkcijski model smo po več fazah testiranja izbrali konfiguracijo: SGD, stopnja učenja (LR) = 0.01, velikost serije (Batch Size) = 64, število epoh = 30. Čeprav so nekateri modeli z optimizatorjem Adam na papirju kazali hitrejšo konvergenco in nižjo začetno izgubo, so matematični in praktični razlogi za našo končno izbiro sledeči:

Enakomerno in linearno učenje (Odpornost na okoljski šum): SGD nima dinamičnega prilagajanja stopnje učenja za vsak parameter posebej (kot ga ima Adam). Uteži posodablja enakomerno in postopoma. To je prisililo mrežo, da je ignorirala visokofrekvenčni šum v ozadju ter se osredotočila na robustne, globalne matematične značilnosti samega obraza (razdalja med očmi, nosom, struktura čeljusti).

Odpornost na preučenost (Overfitting): Razlika med učno in validacijsko izgubo je pri tej konfiguraciji minimalna. To dokazuje, da model odlično posplošuje znanje (generalization) in je imun na spremembe okolja.

Uravnoteženost paketa (Batch Size = 64): Velikost paketa 32 je vnašala preveč statističnega šuma (cik-cak grafi), kar je podaljševalo čas stabilizacije uteži. Paket 128 je na drugi strani preveč zgladil gradient, da bi model v omejenem času dosegel polni potencial. Vrednost 64 predstavlja idealno zlato sredino – ponuja stabilen gradient za enakomerno konvergenco in hkrati ravno pravšnjo mero stohastičnega šuma, ki deluje kot naravna regularizacija.

Uglasitev z 30 epohami: SGD zaradi svoje linearne narave potrebuje več časa za "ogrevanje uteži" in premik zamrznjenih filtrov ResNet18. Pri 12 epohah je bil model še matematično podtreniran (underfitted) – zaupanje v živo je znašalo le okoli 46 %. Pri 20 epohah je zaupanje zraslo na približno 60 %, kar je še vedno povzročalo, da je aplikacija odobrila dostop šele v 4. zaporednem poskusu. Šele pri 30 epohah je krivulja izgube dosegla stabilen plato, ne da bi validacijska izguba začela ponovno rasti. Model je dokončno izpilil biometrične meje med razredi, s čimer je surovo zaupanje v živo varno poskočilo nad 85 %.

4. Praktični primer: Zakaj je bila sprememba nujna v realnem svetu?

Da bi lažje razumeli, zakaj je matematična teorija s papirja v praksi pomenila varnostno katastrofo ali uporabniško blokado, si oglejmo realni produkcijski scenarij v treh različnih stopnjah razvoja sistema.

Scenarij v živo: Razvijalec A se želi prijaviti v Android aplikacijo ob 20. uri zvečer

SITUACIJA 1: Uporaba modela z optimizatorjem Adam (Navideti 100-% ideal iz Faze 1). Razvijalec A odpre aplikacijo. Ker je večer, je svetloba v sobi slabša, v ozadju pa se vidi bela omara. Model Adam v ozadju analizira sliko. Ker se je med učenjem preučil (overfitted) na okolje, sploh ne gleda obraza Razvijalca A. Namesto tega v bazi išče, kdo se je slikal pred belo omaro ob slabši svetlobi. Ugotovi, da ima Razvijalec B v učni množici tri slike z natanko takšnim ozadjem. Model vrne napoved: "To je Razvijalec B" z lažno 99.9% samozavestjo.

Rezultat: Aplikacija izpiše "Živijo, Razvijalec B", zlorabi pravice in odpre napačen predal paketnika. Varnost sistema je popolnoma kompromitirana zaradi prehitrega in površinskega učenja optimizatorja Adam.

SITUACIJA 2: Uporaba prvega SGD modela (12 epoh – Podtreniranost). Razvijalec A stopi pred kamero v istem okolju. Stabilni SGD model pravilno ignorira omaro in svetlobo ter se osredotoči izključno na njegove obrazne poteze. Pravilno ugotovi, da je to Razvijalec A. Vendar, ker je model treniral le 12 epoh, so njegove meje odločanja še zelo "previdne" in matematično nerazvite. Model reče: "Mislim, da je to Razvijalec A, vendar sem prepričan le 46.78%." Ker je bil prvotni varnostni prag v FastAPI nastavljen visoko (0.65), aplikacija javi verified = false.

Rezultat: Sistem pravega uporabnika zavrne in se zacikla v neskončno ponavljanje. Varnost je sicer zagotovljena (napačnih oseb ne spušča skozi), vendar je uporabniška izkušnja popolnoma uničena, saj sistem v živo ne deluje.

SITUACIJA 3: Končni zmagovalni model (SGD, 30 epoh, Batch Size 64, LR 0.01). Razvijalec A ponovno odpre aplikacijo v enakih pogojih. Končni model, ki je skozi 30 epoh natančno izpilil biometrične robove, izolira obrazne poteze in popolnoma ignorira svetlobo ter belo omaro. Zaradi zadostnega števila učnih ciklov (30 epoh) so uteži idealno nastavljene. Model javi: "To je Razvijalec

A, stopnja zaupanja je 88.50 %." FastAPI aplikacija primerja rezultat z visokim varnostnim pragom (CONFIDENCE_THRESHOLD = 0.80). Pogoj je uspešno izpolnjen ($0.885 > 0.80$).

Rezultat: Aplikacija v sekundi nemoteno preusmeri uporabnika na MainActivity in varno odpre pravi paketnik že v prvem poskusu.

5. Integracija z Android aplikacijo in odprava produkcijskih hroščev

Izbira stabilnejšega SGD modela in optimizacija kode sta neposredno rešili tri kritične težave v produkcijskem okolju:

Odprava UnboundLocalError na strežniku: Prvotna predobdelava slik (pripravi_sliko_za_api) je ob neprepoznanem obrazu povzročila sesutje API-ja, saj spremenljivka obraz_najden v else veji ni bila definirana. Z novo logiko funkcija sedaj vedno vrne par (slika, status), kar omogoča nemoteno delovanje strežnika.

Tekoč uporabniški vmesnik: Novi model omogoča takojšnjo izvedbo: ob uspehu (verified == true) aplikacija uporabnika v sekundi preusmeri na MainActivity, ob neuspehu spodleti tiho in sproži avtomatiziran ponovni poskus čez 2 sekundi.

Preprečitev nepooblaščenih vstopov: S starim modelom se je dogajalo, da se je s svojim obrazom prijavil Razvijalec A, aplikacija pa je izpisala "Živijo, Razvijalec B" in odpirala paketnik z napačnimi pravicami. Novi SGD model z daljšim časom učenja je tovrstne biometrične zamenjave popolnoma odpravil.